

Understanding the Jacobian Matrix

<https://gemini.google.com/app/ac39b331150c19e1>

User prompt: jacobian matrix?

Response: The **Jacobian matrix** is a matrix of all first-order partial derivatives of a vector-valued function. It represents the best linear approximation of a multivariable function near a given point—essentially acting as the multidimensional equivalent of the single-variable derivative.

For a function $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ that takes an n -dimensional input vector $\mathbf{x} = (x_1, x_2, \dots, x_n)$ and outputs an m -dimensional vector $\mathbf{f}(\mathbf{x}) = (f_1, f_2, \dots, f_m)$, the Jacobian matrix $\mathbf{J}$ is an $m \times n$ matrix defined as:

$$\mathbf{J} = \frac{\partial(f_1, \dots, f_m)}{\partial(x_1, \dots, x_n)} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Step-by-Step Example: Polar to Cartesian Coordinates

Consider converting polar coordinates (r, θ) to 2D Cartesian coordinates (x, y) :

$$f_1(r, \theta) = x = r \cos \theta$$

$$f_2(r, \theta) = y = r \sin \theta$$

1. Calculate the partial derivatives:

- $\frac{\partial x}{\partial r} = \cos \theta$
- $\frac{\partial x}{\partial \theta} = -r \sin \theta$
- $\frac{\partial y}{\partial r} = \sin \theta$
- $\frac{\partial y}{\partial \theta} = r \cos \theta$

2. Assemble the matrix:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial \theta} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{bmatrix}$$

3. Compute the Jacobian Determinant: When $m = n$, the matrix is square, and its determinant $\det(\mathbf{J})$ scales area or volume local changes:

$$\det(\mathbf{J}) = (\cos \theta)(r \cos \theta) - (-r \sin \theta)(\sin \theta) = r(\cos^2 \theta + \sin^2 \theta) = r$$

This factor r is precisely the area element scaling factor $dx dy = r dr d\theta$ used in multivariable integration.

Primary Applications

- **Robotics (Inverse Kinematics):** Maps joint velocities to linear and angular velocities of the robot's end-effector.
- **Multivariable Calculus:** The determinant $\det(\mathbf{J})$ serves as the scaling factor (distortion ratio) during a change of variables in double or triple integrals.
- **Machine Learning & Neural Networks:** Used in calculating sensitivities for backpropagation and optimizing non-linear systems via methods like Gauss-Newton or Levenberg-Marquardt.
- **Differential Equations & Dynamical Systems:** Used to linearize non-linear systems around fixed points to determine local stability.

User prompt: $\mathbf{J} = \frac{\partial (f_1, \dots, f_m)}{\partial (x_1, \dots, x_n)} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$ Why row 1 is about f_1 , why not column 1?

Response: Row 1 contains all partial derivatives of f_1 so that the standard linear approximation formula works using column vectors:

$$d\mathbf{f} = \mathbf{J} d\mathbf{x}$$

1. Compatibility with Matrix Multiplication

In multivariable calculus, inputs and outputs are typically represented as column vectors:

$$d\mathbf{x} = \begin{bmatrix} dx_1 \\ dx_2 \\ \vdots \\ dx_n \end{bmatrix}_{(n \times 1)} \quad \text{and} \quad d\mathbf{f} = \begin{bmatrix} df_1 \\ df_2 \\ \vdots \\ df_m \end{bmatrix}_{(m \times 1)}$$

For $d\mathbf{f} = \mathbf{J} d\mathbf{x}$ to hold, $\mathbf{J}$ must have dimension $m \times n$. To calculate the total differential of the first output component df_1 , the multi-variable chain rule states:

$$df_1 = \frac{\partial f_1}{\partial x_1} dx_1 + \frac{\partial f_1}{\partial x_2} dx_2 + \cdots + \frac{\partial f_1}{\partial x_n} dx_n$$

In matrix multiplication, computing the first entry of the output vector (df_1) requires taking the **dot product of Row 1 of $\mathbf{J}$ with the column vector $d\mathbf{x}$** :

$$\begin{bmatrix} df_1 \\ \vdots \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix} \begin{bmatrix} dx_1 \\ dx_2 \\ \vdots \\ dx_n \end{bmatrix}$$

2. Stacking Gradients of Component Functions

The gradient of a single scalar function f_i represented as a row vector (a 1-form) is:

$$\nabla f_i = \left[\frac{\partial f_i}{\partial x_1} \quad \frac{\partial f_i}{\partial x_2} \quad \cdots \quad \frac{\partial f_i}{\partial x_n} \right]$$

The Jacobian matrix is simply formed by stacking the gradient row vectors of each component function on top of one another:

$$\mathbf{J} = \begin{bmatrix} \nabla f_1 \\ \nabla f_2 \\ \vdots \\ \nabla f_m \end{bmatrix}$$

3. What If Column 1 Was f_1 ?

If row 1 and column 1 were swapped, the resulting matrix would be the **transpose $\mathbf{J}^T$** (often called *denominator layout* notation in matrix calculus). With that layout:

- You would have to write the linear approximation using row vectors instead: $d\mathbf{f}^T = d\mathbf{x}^T \mathbf{J}^T$.
- Or perform matrix multiplication as $d\mathbf{f} = \mathbf{J}^T d\mathbf{x}$ which fails dimension matching ($(n \times m) \times (n \times 1)$ is invalid when $m \neq n$).

The standard convention (*numerator layout*) ensures f_i corresponds to row i and x_j corresponds to column j , maintaining consistency across linear algebra and calculus.

User prompt: what about derivative of a 2 by matrix with respect to a 2 by 3 matrix?

Response: Differentiating a 2×2 **matrix** with respect to a 2×3 **matrix** results in a **4th-order tensor** containing $2 \times 2 \times 2 \times 3 = 24$ total partial derivatives.

Since standard 2D paper/screens cannot display a 4D grid directly, mathematicians and machine learning frameworks handle this in one of two ways:

Option 1: Flattening into Vectors (The Jacobian Approach)

You flatten both matrices into vectors using column-major or row-major order:

1. Flatten Y (2×2) into a 4D column vector $\text{vec}(Y)$.
2. Flatten X (2×3) into a 6D column vector $\text{vec}(X)$.

The derivative $\frac{\partial \text{vec}(Y)}{\partial \text{vec}(X)}$ is a standard 4×6 **Jacobian matrix**:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial Y_{11}}{\partial X_{11}} & \frac{\partial Y_{11}}{\partial X_{12}} & \frac{\partial Y_{11}}{\partial X_{13}} & \frac{\partial Y_{11}}{\partial X_{21}} & \frac{\partial Y_{11}}{\partial X_{22}} & \frac{\partial Y_{11}}{\partial X_{23}} \\ \frac{\partial Y_{12}}{\partial X_{11}} & \frac{\partial Y_{12}}{\partial X_{12}} & \frac{\partial Y_{12}}{\partial X_{13}} & \frac{\partial Y_{12}}{\partial X_{21}} & \frac{\partial Y_{12}}{\partial X_{22}} & \frac{\partial Y_{12}}{\partial X_{23}} \\ \frac{\partial Y_{21}}{\partial X_{11}} & \frac{\partial Y_{21}}{\partial X_{12}} & \frac{\partial Y_{21}}{\partial X_{13}} & \frac{\partial Y_{21}}{\partial X_{21}} & \frac{\partial Y_{21}}{\partial X_{22}} & \frac{\partial Y_{21}}{\partial X_{23}} \\ \frac{\partial Y_{22}}{\partial X_{11}} & \frac{\partial Y_{22}}{\partial X_{12}} & \frac{\partial Y_{22}}{\partial X_{13}} & \frac{\partial Y_{22}}{\partial X_{21}} & \frac{\partial Y_{22}}{\partial X_{22}} & \frac{\partial Y_{22}}{\partial X_{23}} \end{bmatrix}$$

Option 2: Tensor Notation (Einstein Index Notation)

If you maintain the multidimensional structure, each partial derivative is indexed by 4 variables:

$$T_{i,j,k,l} = \frac{\partial Y_{i,j}}{\partial X_{k,l}}$$

where:

- $i \in \{1, 2\}$ and $j \in \{1, 2\}$ (indexing the 2×2 output Y)
- $k \in \{1, 2\}$ and $l \in \{1, 2, 3\}$ (indexing the 2×3 input X)

Concrete Example

Suppose $Y = XA$, where:

- X is a 2×3 matrix
- A is a fixed 3×2 matrix

Then Y is a 2×2 matrix:

$$Y_{ij} = \sum_{m=1}^3 X_{im} A_{mj}$$

Taking the derivative of entry Y_{ij} with respect to entry X_{kl} :

$$\frac{\partial Y_{ij}}{\partial X_{kl}} = \delta_{ik} A_{lj}$$

(where $\delta_{ik} = 1$ if $i = k$, and 0 otherwise)

If you write this out in flattened matrix form $\mathbf{J}$, it yields the **Kronecker product** $A^T \otimes I_2$, which is a 4×6 matrix.

This [video on Computing derivatives of matrix and tensor expressions](#) breaks down how matrix and higher-order tensor derivatives are formulated and processed in computational frameworks.

Sources

1. [Computing derivatives of matrix and tensor expressions - YouTube](#)

User prompt: Z is a 2×1 matrix and W is a 2×3 matrix. Z is the weighted sum matrix of a neural layer with 2 neuron, and W is the weight matrix of that layer. So, conceptually, row-by-row derivative is enough for me.

Response: Breaking down Z row-by-row works cleanly because each row z_i corresponds to a distinct neuron in the layer.

In your setup, since W is 2×3 , the input vector X must be 3×1 :

$$Z = WX + b \implies \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

Row-by-Row Breakdown

1. Neuron 1 (Row 1):

$$z_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1$$

The derivative of z_1 with respect to its own weights (Row 1 of W) is:

$$\frac{\partial z_1}{\partial W_1} = \begin{bmatrix} \frac{\partial z_1}{\partial w_{11}} & \frac{\partial z_1}{\partial w_{12}} & \frac{\partial z_1}{\partial w_{13}} \end{bmatrix} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} = X^T$$

The derivative of z_1 with respect to Neuron 2's weights (Row 2 of W) is zero:

$$\frac{\partial z_1}{\partial W_2} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

2. Neuron 2 (Row 2):

$$z_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2$$

Similarly, z_2 only depends on Row 2 of W :

$$\frac{\partial z_2}{\partial W_1} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}, \quad \frac{\partial z_2}{\partial W_2} = X^T$$

How This Works in Backpropagation

In neural networks, you rarely compute $\frac{\partial Z}{\partial W}$ in isolation. Instead, backpropagation gives you the gradient of a scalar loss L with respect to Z :

$$\frac{\partial L}{\partial Z} = \begin{bmatrix} \frac{\partial L}{\partial z_1} \\ \frac{\partial L}{\partial z_2} \end{bmatrix} = \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix}$$

Using your row-by-row breakdown via the chain rule for each weight row W_i :

- **Row 1 gradient:** $\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial W_1} = \delta_1 X^T = \begin{bmatrix} \delta_1 x_1 & \delta_1 x_2 & \delta_1 x_3 \end{bmatrix}$
- **Row 2 gradient:** $\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial W_2} = \delta_2 X^T = \begin{bmatrix} \delta_2 x_1 & \delta_2 x_2 & \delta_2 x_3 \end{bmatrix}$

Stacking these rows back together gives the familiar matrix backpropagation rule:

$$\frac{\partial L}{\partial W} = \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix} \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} = \frac{\partial L}{\partial Z} X^T$$

This outer product matches the shape of W (2×3) directly, completely bypassing 4D tensors.

User prompt: Such corresponding row-row derivative has a name like jacobian?

Response: There isn't a single universal name like "Jacobian" specifically for the row-by-row derivative of a vector with respect to a matrix. Instead, math and machine learning literature use a few terms depending on the perspective:

1. Vector-by-Row Gradient (or Row-wise Gradient)

In machine learning notation, taking the derivative of a single output component z_i with respect to a row vector W_i is simply the **gradient of z_i with respect to W_i** :

$$\nabla_{W_i} z_i = \frac{\partial z_i}{\partial W_i}$$

Because z_i is a scalar and W_i is a row vector (1×3), this gives a standard row vector gradient (1×3).

2. A Row Slice of the 3D Jacobian Tensor

Strictly speaking, the full derivative of a vector Z (2×1) with respect to a matrix W (2×3) is a **3D Jacobian Tensor J** of shape $2 \times 2 \times 3$.

What you are doing row-by-row corresponds to looking at specific **slices or blocks** of this tensor:

- $J_{i,j,k} = \frac{\partial z_i}{\partial W_{jk}}$
- Because neuron i only depends on row i of the weights, the tensor is **sparse**: $J_{i,j,k} = 0$ whenever $i \neq j$.

3. Block-Diagonal Jacobian (or Layer-wise Jacobian)

When you combine all those row derivatives into a single flattened standard matrix, it is referred to as a **Block-Diagonal Jacobian**.

If you flatten W into a 6×1 vector, the actual 2×6 Jacobian matrix looks like this:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial z_1}{\partial W_1} & \mathbf{0} \\ \mathbf{0} & \frac{\partial z_2}{\partial W_2} \end{bmatrix} = \begin{bmatrix} x_1 & x_2 & x_3 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & x_2 & x_3 \end{bmatrix}$$

Because of the block structure (where the off-diagonal blocks are zero), computing the full 2×6 Jacobian matrix is unnecessary in practice. Computing the **row-wise gradients** directly skips all the useless zero computations.

User prompt: How do autograd frameworks like PyTorch or TensorFlow compute gradients without building 3D Jacobian tensors?

Response: Autograd engines like PyTorch, JAX, and TensorFlow bypass computing or storing intermediate multi-dimensional Jacobian tensors by working exclusively with **Vector-Jacobian Products (VJPs)**.

Instead of materializing large high-dimensional partial derivative matrices, they perform localized matrix-vector or matrix-matrix operations backwards along the dynamic execution graph.

1. The Core Insight: You Always End Up with a Scalar Loss

In deep learning, you almost never need the standalone Jacobian $\frac{\partial Z}{\partial W}$. You only need the gradient of a single scalar loss value L with respect to the parameters W :

$$\frac{\partial L}{\partial W}$$

By applying the chain rule, this gradient is formed as:

$$\frac{\partial L}{\partial W} = \underbrace{\frac{\partial L}{\partial Z}}_{\mathbf{v}^T} \cdot \underbrace{\frac{\partial Z}{\partial W}}_{\mathbf{J}}$$

Notice that $\frac{\partial L}{\partial Z}$ is a vector (or matrix matching the shape of Z). The autograd engine never computes $\mathbf{J}$ on its own. Instead, it directly computes the product $\mathbf{v}^T \mathbf{J}$ (the Vector-Jacobian Product).

2. How VJP Avoids Higher-Order Tensors

Consider a linear layer $Z = WX$ where:

- X is 3×1
- W is 2×3
- Z is 2×1

If you calculated $\mathbf{J} = \frac{\partial Z}{\partial \mathbf{W}}$ explicitly, it would be a **3D tensor** of shape $(2 \times 2 \times 3)$ containing 12 values (mostly zeros).

However, during backpropagation, the engine receives the incoming upstream gradient from the loss:

$$\mathbf{v} = \frac{\partial L}{\partial Z} = \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix} \quad (2 \times 1 \text{ vector})$$

Instead of building a 3D tensor, the autograd module for the matrix multiplication node executes its predefined **VJP rule**:

$$\text{VJP}_W(\mathbf{v}) = \mathbf{v}X^T = \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix} \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} = \begin{bmatrix} \delta_1 x_1 & \delta_1 x_2 & \delta_1 x_3 \\ \delta_2 x_1 & \delta_2 x_2 & \delta_2 x_3 \end{bmatrix}$$

This produces a 2×3 **matrix** (matching W 's shape) via a simple outer product, skipping the 3D tensor calculation entirely.

3. How Autograd Executes This Under the Hood

When PyTorch executes a forward pass, it constructs a **Dynamic Computational Graph (DAG)**:

- 1. Forward Pass:** Every operation (like `torch.matmul` or `torch.relu`) creates a node. The node saves *only* the minimum inputs/outputs required for backprop (e.g., saving X for matrix multiplication).
- 2. Backward Pass:** Calling `loss.backward()` passes a scalar seed ($\mathbf{v} = 1.0$) back through the root node.
- 3. Graph Traversal:** Each node receives an incoming vector $\mathbf{v}$ from its successor and executes a custom primitive function that outputs $\mathbf{vJ}$ directly to pass upstream.

Summary Comparison

Metric / Step	Explicit Jacobian Approach	Autograd's VJP Approach
Intermediate Object	3D/4D Jacobian Tensor	Scalar/Vector values propagated backwards
Memory Complexity	$O(N \cdot M)$ memory blowout	$O(N)$ (proportional to model memory)

Metric / Step	Explicit Jacobian Approach	Autograd's VJP Approach
Operation Type	Multi-index tensor contractions	Standard linear algebra (<code>matmul</code> , outer products)

Sources

1. [A new trick for calculating Jacobian vector products](#)
-

User prompt: I need a theoretical understanding.

Response: To build a rigorous theoretical understanding of how calculus, linear algebra, and autograd intersect, we have to look at the **Differential of a Function** and the **Adjoint of a Linear Operator**.

1. Derivatives as Linear Transformations (Fréchet Derivative)

In rigorous multivariable calculus, a function $F : V \rightarrow W$ between two vector spaces (or matrix spaces) is differentiable at X if there exists a **linear operator** $DF(X) : V \rightarrow W$ such that:

$$F(X + H) = F(X) + [DF(X)](H) + o(\|H\|)$$

Notice the core concept:

- $DF(X)$ is **not a matrix or a tensor**—it is a **linear map** (a function that takes a variation H and returns a variation ΔY).
- If $X \in \mathbb{R}^{2 \times 3}$ and $Y \in \mathbb{R}^{2 \times 1}$, the linear map $DF(X)$ takes an input of shape 2×3 and returns an output of shape 2×1 .

For the linear layer $Z(W) = WX$:

$$Z(W + H) = (W + H)X = WX + HX$$

The linear differential map evaluated on a perturbation matrix H is simply:

$$[DZ(W)](H) = HX$$

2. Dual Spaces and the Trace Inner Product

To compute gradients of a scalar loss function $L(Z)$, we define an inner product on matrix spaces. For matrices, the standard inner product is the **Frobenius inner product**:

$$\langle A, B \rangle = \text{tr}(A^T B)$$

The scalar differential of the loss function $L : \mathbb{R}^{2 \times 1} \rightarrow \mathbb{R}$ with respect to Z is represented by its gradient $\nabla_Z L$ via Riesz Representation:

$$dL = \langle \nabla_Z L, dZ \rangle = \text{tr}((\nabla_Z L)^T dZ)$$

3. The Theoretical Derivation of the Backprop Rule

To find the gradient with respect to W ($\nabla_W L$), we use the chain rule for differentials:

$$dL = \langle \nabla_Z L, dZ \rangle$$

Substitute the differential $dZ = [DZ(W)](dW) = dW X$:

$$dL = \text{tr}((\nabla_Z L)^T (dW X))$$

Using the cyclic property of the trace ($\text{tr}(ABC) = \text{tr}(CAB)$):

$$dL = \text{tr}(X(\nabla_Z L)^T dW) = \text{tr}(((\nabla_Z L)X^T)^T dW)$$

By definition of the inner product $\langle \nabla_W L, dW \rangle = \text{tr}((\nabla_W L)^T dW)$, we can directly extract $\nabla_W L$:

$$\nabla_W L = (\nabla_Z L)X^T$$

4. The Operator Adjoint Perspective

In functional analysis, the **Adjoint** of a linear operator $A : V \rightarrow W$ is the operator $A^* : W \rightarrow V$ satisfying:

$$\langle A(v), w \rangle_W = \langle v, A^*(w) \rangle_V$$

In backpropagation:

1. The forward differential is the operator $T(H) = HX$.
2. The backpropagation step is applying the **adjoint operator** T^* to the incoming loss gradient $\mathbf{v} = \nabla_Z L$:

$$\langle T(H), \mathbf{v} \rangle = \langle HX, \mathbf{v} \rangle = \text{tr}(\mathbf{v}^T HX) = \text{tr}(X\mathbf{v}^T H) = \langle H, \mathbf{v}X^T \rangle$$

Thus, the adjoint operator is:

$$T^*(\mathbf{v}) = \mathbf{v}X^T$$

Summary of Theoretical Correspondence

- **Multivariable Calculus View:** A derivative is a **Linear Map** $DF(X)$, not an array of numbers.
 - **Autograd / Adjoint View:** Reverse-mode automatic differentiation is mathematically identical to computing the **Adjoint Linear Operator** $(DF(X))^*$.
 - **Computational Advantage:** Evaluated on a scalar seed from the loss, the adjoint map maps directly back from the co-domain shape to the domain shape without ever requiring higher-order tensor coordinates.
-

User prompt: No, I meant about the row-wise gradient

Response: The theoretical foundation of the **row-wise gradient** comes down to **block-structure decoupling** in differential calculus.

When a vector-valued function $Z(W) = WX + b$ acts on a matrix parameter W , the mathematical transformation physically separates across the rows of W .

1. Vector Space Decomposition (Direct Sums)

To formalize this, consider the space of $m \times n$ matrices $\mathbb{R}^{m \times n}$. Mathematically, this space can be decomposed into a **direct sum of row vector spaces**:

$$\mathbb{R}^{m \times n} = W_1 \oplus W_2 \oplus \cdots \oplus W_m$$

where each $W_i \in \mathbb{R}^{1 \times n}$ represents the i -th row of W .

Because the i -th scalar output z_i depends **exclusively** on the i -th row subspace W_i , the multivariable function $Z(W)$ exhibits complete **orthogonal independence** between its outputs and unaligned inputs:

$$\frac{\partial z_i}{\partial W_j} = \mathbf{0}_{1 \times n} \quad \text{for all } i \neq j$$

2. Block-Diagonal Projection of the Tangent Map

In differential geometry, the derivative of a map $Z : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^m$ at point W is a linear pushforward map between tangent spaces:

$$dZ_W : T_W(\mathbb{R}^{m \times n}) \rightarrow T_{Z(W)}(\mathbb{R}^m)$$

Because of the direct sum decomposition, the matrix representation of this differential operator under flattened bases exhibits a **block-diagonal structure**:

$$\mathbf{J} = \begin{bmatrix} \nabla_{W_1} z_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \nabla_{W_2} z_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \nabla_{W_m} z_m \end{bmatrix} = \bigoplus_{i=1}^m \nabla_{W_i} z_i$$

The direct sum symbol $\bigoplus$ captures the core theory: the total global derivative isn't an arbitrary high-dimensional structure; it is the **direct sum of independent row-wise gradients**.

3. Coordinate-Free Proof via the Row Linear Map

For a single neuron i , the operation is a bilinear map $z_i(W_i, X) = \langle W_i^T, X \rangle$.

Taking the directional derivative (Gâteaux derivative) of z_i in the direction of an arbitrary perturbation row $H_i \in \mathbb{R}^{1 \times n}$:

$$\begin{aligned} dz_i(W_i; H_i) &= \lim_{\epsilon \rightarrow 0} \frac{z_i(W_i + \epsilon H_i) - z_i(W_i)}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0} \frac{((W_i + \epsilon H_i)X + b_i) - (W_i X + b_i)}{\epsilon} = H_i X \end{aligned}$$

By the Riesz Representation Theorem, the gradient $\nabla_{W_i} z_i$ is defined as the unique vector satisfying $dz_i(W_i; H_i) = \langle \nabla_{W_i} z_i, H_i \rangle$.

Since $H_i X = \text{tr}(H_i X) = \text{tr}(X H_i) = \langle X^T, H_i \rangle$:

$$\nabla_{W_i} z_i = X^T$$

Summary of Theoretical Properties

- **Decoupling (Sparsity):** The $m \times n$ parameters do not interact globally during the forward linear transformation. They split into m independent 1D sub-problems.
 - **Direct Sum Operator:** The total differential map dZ is the direct sum of independent row differentials: $dZ = \bigoplus_{i=1}^m dz_i$.
 - **Gradient Isolation:** Computing $\nabla_{W_i} z_i$ isolates the i -th slice of the dual space (cotangent bundle), rendering the cross-row partial derivatives $\frac{\partial z_i}{\partial W_j}$ identically zero.
-